UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias · Semestre 2027-I

FUNDAMENTOS DE BASES DE DATOS

Práctica 06

Mantenimiento de llaves foráneas

Clientes, tarjetas y partidas

Equipo ÑIPGG

NOMBRE COMPLETO	NÚMERO DE CUENTA
Omar Alejandro Juárez Larrondo	322245244
Yahir León Bautista	322176542
Diego Hernández Gómez	321069942
Miguel Ángel Jiménez Ramírez	119000887
Ana Lilia Carballido Camacateco	315314601

Profesor: Gerardo Avilés Rosas

2 de noviembre de 2026

Mantenimiento de llaves foráneas: NO ACTION

1. ¿Qué es una política de mantenimiento de llaves foráneas?

Es la acción referencial que una restricción FOREIGN KEY define para mantener la relación entre una fila hija y su fila referenciada cuando esta última se elimina o cambia su clave. Se especifican por separado ON DELETE y ON UPDATE. La acción corresponde a operaciones sobre las filas, no a DROP TABLE. Eliminar una tabla tiene reglas de dependencias distintas.

En este esquema, Partida referencia la pareja (idCliente, numeroTarjeta) de Tarjeta. La integridad exige que esa pareja exista completa. La política decide qué ocurre al borrar la tarjeta o corregir alguno de esos componentes. No basta con que cada componente aparezca por separado en tarjetas diferentes.

PostgreSQL ofrece NO ACTION, RESTRICT, CASCADE, SET NULL y SET DEFAULT. No es necesario elegir una única acción para toda la base: la decisión depende de la relación y de si se está borrando o actualizando. Las acciones no permiten insertar referencias inexistentes. [1]

2. ¿Cómo se indica NO ACTION en SQL?

```
ALTER TABLE "Partida" DROP CONSTRAINT fk_partida_tarjeta;
ALTER TABLE "Partida" ADD CONSTRAINT fk_partida_tarjeta
    FOREIGN KEY ("idCliente", "numeroTarjeta")
    REFERENCES "Tarjeta" ("idCliente", "numeroTarjeta")
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
    NOT DEFERRABLE;
```

El ejemplo ilustra ambas operaciones con NO ACTION; no es la elección final de UPDATE para Partida. En mi implementación utilizo ON DELETE NO ACTION y ON UPDATE CASCADE. Cuando se omiten las acciones, el valor predeterminado es NO ACTION; las escribo explícitamente para que el criterio sea revisable. [1]

3. Objeto y funcionamiento

NO ACTION conserva la comprobación de la referencia y rechaza la operación si, al comprobar la FK, quedan hijos sin padre. No borra hijos ni les asigna un valor alternativo. NOT DEFERRABLE es el comportamiento de las FK de mi módulo: la violación no puede mantenerse entre sentencias para repararla más tarde.

Una FK con NO ACTION puede declararse DEFERRABLE. Si se difiere, se permite reparar una inconsistencia temporal antes de comprobarla; una inconsistencia que persiste no puede confirmarse. RESTRICT no permite posponer su acción sobre el padre. No se debe confundir NO ACTION con ignorar la integridad. [1][2]

Ejemplo reproducible de comprobación diferida

Ejecutar esta secuencia en una sesión de prueba. Son tablas temporales, independientes del modelo de negocio.

```
BEGIN;
CREATE TEMP TABLE padre_demo(id integer PRIMARY KEY);
CREATE TEMP TABLE hijo_demo(
    id integer PRIMARY KEY,
    padre integer NOT NULL,
    CONSTRAINT fk_demo FOREIGN KEY(padre) REFERENCES padre_demo(id)
        ON DELETE NO ACTION ON UPDATE NO ACTION
        DEFERRABLE INITIALLY IMMEDIATE
);
INSERT INTO padre_demo VALUES (1);
INSERT INTO hijo_demo VALUES (1, 1);

-- Sin diferir, cambiar 1 por 2 en el padre viola fk_demo.
SET CONSTRAINTS fk_demo DEFERRED;
UPDATE padre_demo SET id = 2 WHERE id = 1;
UPDATE hijo_demo SET padre = 2 WHERE padre = 1;
SET CONSTRAINTS fk_demo IMMEDIATE; -- La pareja reparada es válida.
SELECT p.id, h.padre FROM padre_demo p JOIN hijo_demo h ON h.padre=p.id;
-- Resultado: 2 | 2.
ROLLBACK;
```

Las pruebas del módulo ejecutan además el caso inmediato, que rechaza el cambio, y una FK RESTRICT declarada diferible, cuya acción sigue rechazándolo.

4. Ventajas y desventajas

Ventajas. En las relaciones que decido proteger, impide borrar el padre mientras se conserve historial asociado. Hace visible la necesidad de resolver las dependencias antes de borrar. No elimina saldo, puntos ni partidas como efecto secundario. Cuando se requiere una corrección coordinada de claves, NO ACTION con una FK diferible permite organizarla dentro de una transacción.

Desventajas. El borrado puede requerir intervención explícita sobre las referencias y fallar al existir dependencias indirectas. Usarlo también en UPDATE obliga a coordinar manualmente las correcciones de claves; por eso elijo CASCADE en UPDATE. Diferir la comprobación introduce estados temporales que requieren disciplina transaccional. Una FK tampoco evita que alguien elimine explícitamente el historial: esa protección adicional corresponde a permisos y reglas de negocio.

Compatibilidad y aplicación

Las columnas del módulo son NOT NULL y no tienen DEFAULT. NO ACTION y CASCADE son compatibles con ese diseño. SET NULL chocaría con la obligatoriedad; SET DEFAULT no proporciona un padre válido por sí mismo. No cambio dominios, claves ni nombres de la práctica anterior. Las seis sustituciones se hacen con ALTER TABLE dentro de una transacción.

Los contactos usan DELETE CASCADE porque son dependientes del cliente; Tarjeta y Partida usan DELETE NO ACTION porque la tarjeta conserva recursos y la partida registra actividad. Las seis FK usan UPDATE CASCADE para mantener sus referencias ante correcciones de identificadores. La decisión por relación se presenta en la segunda parte.

Documentación con COMMENT

Los comentarios se guardan como metadatos de PostgreSQL con COMMENT ON TABLE, COMMENT ON COLUMN y COMMENT ON CONSTRAINT. Se aplican después de sustituir las FK, porque borrar una restricción elimina también su comentario. Se comprueban con obj_description y col_description. [3]

En PostgreSQL 18 las restricciones NOT NULL aparecen también en pg_constraint; se comentan las 26, además de las 36 restricciones PK, UNIQUE, CHECK y FK. [4]

Elección y validación de políticas del módulo 01

Alcance y criterio

Implemento las políticas de las seis FK cuya tabla origen pertenece a Cliente, correoElectrónico, teléfono, Tarjeta o Partida. Documento sus cinco tablas, 26 columnas y todas sus restricciones. La base mantiene la estructura de la práctica 05, incluida la unicidad de idCliente en Tarjeta: cada cliente tiene como máximo una tarjeta.

Las acciones siguientes son decisiones de diseño para este módulo: eliminar contactos dependientes, conservar recursos e historial, y propagar correcciones de claves. Ninguna de estas políticas permite insertar hijos huérfanos.

5. Política elegida y motivos por relación

FK	ON DELETE	ON UPDATE
fk_correo_cliente	CASCADE	CASCADE
fk_telefono_cliente	CASCADE	CASCADE
fk_tarjeta_cliente	NO ACTION	CASCADE
fk_tarjeta_cajero	NO ACTION	CASCADE
fk_partida_tarjeta	NO ACTION	CASCADE
fk_partida_maquina	NO ACTION	CASCADE

Contactos: fk_correo_cliente y fk_telefono_cliente

El correo y el teléfono describen a su cliente y no tienen una existencia independiente en el modelo. DELETE CASCADE elimina esos contactos cuando se elimina un cliente sin otras dependencias protegidas. UPDATE CASCADE conserva la asociación si se corrige idCliente, actualizando también la parte de su PK. Descarto NO ACTION para el borrado de contactos porque exigiría una limpieza manual sin aportar conservación de historial. SET NULL rompería NOT NULL y la PK compuesta. No hay un propietario predeterminado al que reasignarlos.

Si el cliente tiene tarjeta, fk_tarjeta_cliente bloquea el borrado completo. La eliminación de contactos no debe interpretarse como una autorización para eliminar al cliente cuando otras FK lo impiden: toda la sentencia se revierte.

Tarjeta: fk_tarjeta_cliente

DELETE NO ACTION impide eliminar al propietario de una tarjeta existente. La tarjeta tiene saldo y puntos, por lo que borrarla en cascada sería una pérdida inadecuada como efecto de borrar al cliente. UPDATE CASCADE propaga idCliente a Tarjeta y, mediante fk_partida_tarjeta, a sus partidas. Mantiene la PK (idCliente, numeroTarjeta) y UNIQUE(idCliente); la corrección a un identificador ya ocupado sigue sujeta a las restricciones de unicidad.

Descarto SET NULL porque idCliente es obligatorio y pertenece a la PK. No existe un cliente predeterminado válido que preserve la propiedad. Un UPDATE NO ACTION haría necesaria la corrección manual coordinada de las referencias.

Tarjeta: fk_tarjeta_cajero

DELETE NO ACTION conserva al cajero identificado como responsable de la venta mientras tenga tarjetas asociadas. Su baja operativa no se representa borrando el registro referenciado. UPDATE CASCADE permite corregir su CURP sin perder esa atribución. Descarto DELETE CASCADE por el riesgo de borrar tarjetas ajenas a la existencia laboral del cajero. CURP es NOT NULL, no tiene DEFAULT y no sería correcto atribuir automáticamente las ventas a otra persona.

Partida: fk_partida_tarjeta

DELETE NO ACTION evita perder partidas por eliminar su tarjeta. UPDATE CASCADE propaga los dos componentes (idCliente, numeroTarjeta), conservando la referencia completa; idMaquina e idPartida no cambian por esa corrección. La PK de Partida sigue siendo (idCliente, numeroTarjeta, idMaquina, idPartida).

Descarto DELETE CASCADE porque las partidas constituyen historial. SET NULL rompería tanto NOT NULL como la PK. Un valor predeterminado no identifica la tarjeta real de la actividad. Probé cambios de numeroTarjeta, de idCliente por la cadena desde Cliente y de ambos componentes simultáneamente.

Partida: fk_partida_maquina

DELETE NO ACTION conserva la referencia al equipo en el que ocurrió la partida. La retirada de una máquina puede representarse en su estado, sin borrar su identidad histórica. UPDATE CASCADE corrige idMaquina también en la PK de Partida. Descarto DELETE CASCADE por la pérdida de historial, y SET NULL o SET DEFAULT por la obligatoriedad y por no existir una máquina sustituta válida.

Compatibilidad con la base y efectos entre módulos

Las seis FK permanecen NOT DEFERRABLE, con nombres, columnas y destinos originales. Todas las columnas siguen siendo obligatorias y sin DEFAULT. Las acciones CASCADE actualizan claves, pero no eliminan la validación de sus CHECK, PK o UNIQUE. Las políticas tampoco implementan una baja lógica ni un sistema de auditoría de cambios de identificadores.

Las FK de VISITA y RECARGA hacia Tarjeta conservan en la base ON UPDATE NO ACTION. Probé que, cuando existen esas filas, una corrección de idCliente se bloquea y revierte la cadena Cliente-Tarjeta-Partida. Es un resultado válido de integridad con la base actual, no una propagación global garantizada. Para permitir esa corrección a través de todo el historial, se debe acordar ON UPDATE CASCADE en las relaciones de VISITA y RECARGA. Esta validación conserva las políticas originales de esas dos tablas.

Otras relaciones también pueden bloquear operaciones: Canje referencia Cliente; RECARGA y los contactos del cajero referencian Cajero; Mantenimiento referencia Máquina. El resultado final depende de todas las referencias existentes. Los casos de mi módulo aíslan cada FK y añaden pruebas explícitas de VISITA y RECARGA.

Resultados y referencias

Verificación ejecutada

Se utilizó PostgreSQL 18.6 en un contenedor aislado y el DDL completo de 30 tablas. El SHA-256 de la base es `588c7910e549284264d0a923dd7a5c6091cd62fd4987063dac388c12a2e99230`.

Resultado: 37 casos correctos, ninguno fallido.

• 24 casos: DELETE y UPDATE del padre, INSERT y UPDATE huérfanos para cada FK.

• 4 casos: cadena Cliente-Tarjeta-Partida, cambio de ambos componentes, referencia compuesta inexistente y borrado de cliente sin tarjeta.

• 2 casos: bloqueo y reversión de la cadena por VISITA y por RECARGA.

• 3 casos: NO ACTION inmediato, NO ACTION diferido reparado y RESTRICT.

• 4 casos de catálogo: políticas de las seis FK y comentarios de 5 tablas, 26 columnas y 62 constraints (incluidos 26 NOT NULL de PostgreSQL 18).

Las operaciones rechazadas comprueban la restricción que causó el error y el estado conservado; las permitidas comprueban las filas resultantes. El conjunto termina con ROLLBACK y verifica que las 30 tablas quedan sin datos de prueba. Las pruebas se ejecutan sobre una base aislada vacía, nunca sobre datos reales.

En PostgreSQL 18, el ejemplo RESTRICT produjo restrict_violation (23001); los rechazos por NO ACTION y las referencias inexistentes produjeron foreign_key_violation (23503). Ambos se manejan sin ocultar errores inesperados.

Fuentes oficiales

Consulta: 2 de noviembre de 2026; documentación correspondiente a PostgreSQL 18.

1. PostgreSQL, §5.5.5, Foreign Keys: acciones referenciales.
https://www.postgresql.org/docs/18/ddl-constraints.html#DDL-CONSTRAINTS-FK

2. PostgreSQL, SET CONSTRAINTS: comprobación inmediata y diferida.
https://www.postgresql.org/docs/18/sql-set-constraints.html

3. PostgreSQL, COMMENT: sintaxis, persistencia y consulta de comentarios.
https://www.postgresql.org/docs/18/sql-comment.html

4. PostgreSQL, pg_constraint: catálogo de restricciones.
https://www.postgresql.org/docs/18/catalog-pg-constraint.html